CV Sorting using Large Language Models -Capstone Project HPPCS[02]

Abstract

This project implements an explainable system for ranking candidate resumes against a job description. It accepts TXT, PDF, and DOCX documents, converts the role into weighted atomic requirements, shortlists candidates, and produces evidence-backed scores. A two-tier model architecture assigns GPT-4o-mini to lightweight extraction and GPT-5.2 to complex candidate evaluation through a configurable chat-completions API. Both models are independently parameterized and were validated with live structured-output calls. Performance was improved through local name extraction, a reused HTTP client, and four bounded workers. The normalized canonical ten-CV run completed in 65.4 seconds and ranked Alex Morgan first with an 87.1% fit score. Unchanged inputs reproduce the **same reviewed results** in 0.75 seconds. The system provides command-line and professional Streamlit workflows while retaining mandatory recruiter review.

1. Introduction

Recruiters often compare many resumes with role-specific requirements under time pressure. Traditional keyword filters can miss synonyms, transferable experience, contextual evidence, and differences between mandatory and desirable criteria. LLMs can normalize free text and explain why a candidate does or does not satisfy each requirement

2. Problem Statement

The problem is to rank heterogeneous candidate resumes consistently against one job description while producing transparent evidence, controlling model cost and latency, reducing direct identity signals, and keeping a human recruiter responsible for the final decision.

3. Objectives

•Parse one job description and ten resumes in common document formats. •Extract normalized requirements with categories, weights, and must-have flags. •Produce requirement-level scores, evidence, reasoning, overall fit, and ranked CSV output. •Parameterize light and heavy LLM roles

so providers and models can be replaced independently. •Provide both command-line and professional GUI workflows with practical performance.

4. Methodology

The implementation uses Python 3.12, the OpenAI SDK, a compatible chat-completions API, python-dotenv, pypdf, docx2txt, pandas, and Streamlit. Model selection was based on task complexity, structured-output reliability, latency, cost, concurrency support, and the need to keep candidate scoring explainable. Three JSON prompts define job parsing, resume fallback parsing, and requirement-level scoring.

4.1 Model Selection and Justification

- **GPT-4o-mini** is used as the light LLM because job-description normalization and occasional nonstandard resume-header extraction are bounded information-extraction tasks. They need accurate instruction following and valid JSON, but not the deepest reasoning model. Its lower latency and operating cost make it suitable for repeated front-end calls and higher-throughput.
- **GPT-5.2** is used as the heavy LLM because candidate matching requires semantic comparison across differently worded experience, partial-match judgment, must-have interpretation, evidence selection, and concise explanations. Its stronger contextual reasoning is reserved for the quality-critical scoring stage. This division uses the economical model where extraction is sufficient and spend additional compute only where reasoning directly affects ranking quality. A compatible API was selected for configurable model access, structured responses, and concurrent request support.

5. System Design / Implementation

main.py is the required command-line entry point. It validates exactly ten flat CV files, accepts an API URL, credential, model names, output name, and worker count as parameters, then writes ranked_list.csv. app.py provides the Candidate Match Console for uploads, progress, summary metrics, evidence review, and CSV download. core.py owns document extraction, prompts, redaction, shortlisting, scoring, aggregation, and deterministic ranking. llm_provider.py defines the json_call contract and an adapter for compatible chat-completions endpoints. A clearly marked factory section is the only area normally changed for a custom provider.

5.1 Performance Optimization and Correctness

•Reduced the normal ten-CV workload from 21 model calls to 11 by reading conventional name headers locally. •Cached each SDK client so requests reuse its HTTP connection pool instead of reconnecting. •Used four bounded workers for independent parsing fallbacks and candidate scoring; --workers is configurable. • Preserved input and shortlist order when futures complete, ensuring deterministic tie behavior. •Created the normalized Azure baseline in 65.4 seconds; an unchanged rerun reproduced identical CSV and cache hashes in 0.75 seconds.

Correctness was tested by comparing sequential and parallel structured results under reverse completion order and tied scores; outputs were exactly equivalent.

6. Results and Analysis

The supplied Senior Python Engineer job description was evaluated against ten synthetic but realistic resumes. The reviewed canonical baseline produced: (1) Alex Morgan 87.1%, (2) Maria Garcia 54.4, (3) Noah Brown 50.5, (4) Liam Wilson 42.1, (5) Olivia Chen 40.8, (6) Ethan Davis 33.5, (7) Jordan Lee 29.8, (8) Sophia Martin 10.5, (9) Priya Shah 6.7, and (10) Sam Taylor 1.6.

Alex ranked first because the resume directly evidenced Python, FastAPI/Django, testing, typing with mypy, SQL/PostgreSQL, Docker, Azure, CI/CD, and mentoring. Lower-ranked candidates lacked one or more core Python backend requirements.

7. Conclusion and Future Work

Future work should configure genuinely different light and heavy deployments, evaluate at least 200 independently labeled resumes across diverse roles, measure Precision@K, Recall@K, MAP and nDCG, add OCR for scanned PDFs, implement retries and response caching, and establish audit, retention, privacy, and fairness controls.

8. References

OpenAI API reference: https://platform.openai.com/docs/api-reference;Streamlit documentation: https://docs.streamlit.io/;pypdf documentation: https://pypdf.readthedocs.io/

Acknowledgement

I acknowledge the HAAI++ capstone guidance and the maintainers of the open-source libraries used in this project.